Василенко Е.Р. 090303-ПИа-о24

Лабораторная работа №17
Разработка асинхронного Telegram-бота на Python с использованием aiogram 3

1. Цель работы

Изучить принципы разработки Telegram-ботов на Python с использованием асинхронного фреймворка aiogram 3 и получить практические навыки: работы с Telegram Bot API, создания асинхронных обработчиков, использования asyncio, организации маршрутизации событий через Router и Dispatcher, обработки команд и текстовых сообщений, применения фильтров, создания Reply Keyboard и Inline Keyboard, обработки Callback Query, использования конечного автомата состояний (FSM) для многошаговых диалогов, работы с middleware, валидации пользовательского ввода, хранения данных в базе данных и разделения приложения на модули.

2. Краткие теоретические сведения

Telegram-бот — программа, взаимодействующая с пользователями через Telegram Bot API. Пользователь отправляет сообщение или нажимает кнопку, Telegram формирует событие Update, приложение через Dispatcher определяет подходящий обработчик. aiogram является асинхронным фреймворком на основе asyncio: вместо последовательной модели ожиданий приложение эффективно переключается между задачами во время ожидания ввода-вывода. Ключевые понятия: coroutine, async, await, event loop, конкурентное выполнение.

Architeorder: Dispatcher — корневой маршрутизатор событий, к нему подключаются Router — логические блоки обработчиков. Обработчики регистрируются через декораторы (@router.message(Command("start")), @router.message(F.text == "...")) с фильтрами. Reply Keyboard заменяет клавиатуру пользователя готовыми кнопками; Inline Keyboard отображается под сообщением, каждая кнопка несёт callback_data (например, book:15), по которому обработчик Callback Query определяет объект.

FSM (Finite State Machine) — конечный автомат состояний для многошаговых диалогов: StatesGroup описывает состояния (State), переходы выполняются через state.set_state(), сохранение данных — state.update_data(), завершение — state.clear(). Команда /cancel позволяет прервать диалог. Middleware выполняет общую логику до/после обработчика (логирование, проверки, измерение времени). Токен бота хранится в переменной окружения, данные — в базе данных (SQLite через SQLAlchemy), бизнес-логика отделена от обработчиков в сервисный слой.

3. Индивидуальное задание

Вариант 5 — предметная область «Отель». Сущности: номер, тип номера, клиент, бронирование. Функции бота: поиск свободных номеров, бронирование, отмена, история бронирований. Требуется реализовать: команды /start, /help, /cancel и дополнительные команды; главное меню (Reply Keyboard) минимум из 4 пунктов; минимум 3 Inline-кнопки с callback data и обработкой Callback Query; минимум один FSM-сценарий (добавление номера, 5 состояний); валидацию минимум 3 ситуаций (некорректная дата, дата выезда раньше заезда, занятый номер, неположительная цена); БД с минимум 3 связанными сущностями и 20 тестовыми записями; CRUD через Telegram; поиск; собственный middleware; разделение на handlers, keyboards, states, services, database, middlewares; полностью асинхронную логику и минимум 10 автоматических тестов.

4. Архитектура приложения

Приложение построено по схеме: Telegram → Dispatcher → Router (handlers) → HotelService → Repository → SQLAlchemy → SQLite, с middleware на пути сообщений. Архитектура приведена на рисунке 17.1.

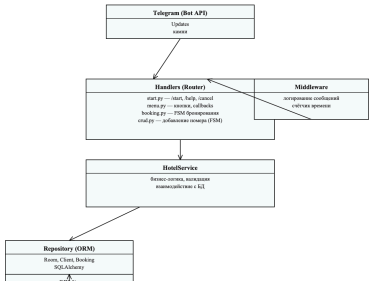


Рисунок 17.1 – Архитектура Telegram-бота

```
lab_17/
├── bot.py                    # точка входа: Bot, Dispatcher, регистрация Router
├── config.py                 # токен и строка БД из переменных окружения
(pydantic-settings)
├── handlers/
│   ├── start.py              # /start, /help, /cancel, /rooms, /stats
│   ├── menu.py               # пункты меню, свободные номера, история, отмена,
поиск
│   ├── booking.py            # FSM-сценарий бронирования (даты)
│   └── crud.py               # FSM-сценарий добавления номера (5 шагов)
├── keyboards/menu.py         # Reply Keyboard и Inline Keyboard (callback data)
├── states/registration.py    # StatesGroup AddRoomStates, BookingStates
├── services/hotel.py         # HotelService — бизнес-логика и валидация
├── database/
│   ├── models.py             # RoomType, Room, Client, Booking + Database
│   ├── repository.py         # репозитории (CRUD, поиск, бронирования)
│   └── seed.py               # заполнение: 3 типа, 20 номеров
├── middlewares/logging.py    # plumbing логирования сообщений
├── tests/                    # 22 теста бизнес-логики
├── requirements.txt
└── .env.example
```

5. Описание сущностей базы данных

База данных hotel_bot.db содержит четыре связанные сущности: RoomType (тип номера), Room (номер), Client (клиент — пользователь Telegram), Booking (бронирование). Связи: RoomType 1:N Room; Room 1:N Booking; Client 1:N Booking. База заполнена тестовыми данными: 3 типа, 20 номеров.

Сущность	Поля	Связи
RoomType	id PK, name UNIQUE	1:N Room
Room	id PK, number UNIQUE, room_type_id FK, price, capacity, status	N:1 RoomType, 1:N Booking
Client	id PK, telegram_id UNIQUE, full_name	1:N Booking
Booking	id PK, client_id FK, room_id FK, check_in, check_out, nights, total	N:1 Client, N:1 Room

Репозитории скрывают SQLAlchemy-запросы: RoomRepository (список, свободные номера с фильтром по вместимости, поиск по номеру или типу, CRUD, смена статуса), ClientRepository (получение/создание клиента по telegram_id), BookingRepository (создание, список клиента, удаление).

6. Router и handlers

В Dispatcher зарегистрированы четыре Router: start (команды), menu (пункты меню и Callback Query), booking (FSM бронирования), crud (FSM добавления номера). Обработчики используют фильтры: Command("start"), Command("help"), F.text == "🏨 Свободные номера", F.data.startswith("book:") и т. д. Callback data имеет структурированный формат: book:<room_id>, cancel:<booking_id>, rtype:<room_type_id>.

Файл: handlers/start.py (фрагмент)

```
from aiogram import Router, F
from aiogram.filters import Command, CommandStart, StateFilter
from aiogram.fsm.context import FSMContext
from aiogram.types import Message

from database import get_db
from database.models import Database
from keyboards.menu import main_menu
from services.hotel import HotelService

router = Router()


def service(db: Database) -> HotelService:
    return HotelService(db.session())


HELP_TEXT = (
    "🏨 <b>Бот гостиницы</b>\n\n"
    "Команды:\n"
    "/start — главное меню\n"
    "/help — помощь\n"
    "/cancel — отменить текущую операцию\n"
    "/rooms — список всех номеров\n"
    "/stats — статистика (админ)\n\n"
    "Возможности меню:\n"
    "— поиск свободных номеров\n"
    "— бронирование номера\n"
    "— отмена бронирования\n"
    "— история бронирований\n"
    "— добавление номера (админ)"
)


@router.message(CommandStart())
async def cmd_start(message: Message, state: FSMContext) -> None:
    await state.clear()
    db = get_db()
    svc = HotelService(db.session())
```

```
    svc.get_or_create_client(
        message.from_user.id,
        message.from_user.full_name or "Клиент",
    )
    await message.answer(
        f"Здравствуйте, {message.from_user.full_name or 'гость'}! "
```

7. Клавиатуры

Главное меню реализовано Reply Keyboard с шестью кнопками: свободные номера, поиск, мои бронирования, добавить номер, отменить бронирование, помощь. Inline-клавиатуры формируются динамически: free_rooms_keyboard создаёт кнопку на каждый свободный номер (callback book:<id>), booking_cancel_keyboard — кнопку на каждое бронирование (callback cancel:<id>), room_types_keyboard — выбор типа номера (callback rtype:<id>).

Файл: keyboards/menu.py

```
from aiogram.types import InlineKeyboardButton, InlineKeyboardMarkup,
KeyboardButton, ReplyKeyboardMarkup


def main_menu() -> ReplyKeyboardMarkup:
    """Reply Keyboard: главное меню бота."""
    return ReplyKeyboardMarkup(
        keyboard=[
            [KeyboardButton(text="🏨 Свободные номера")],
            [KeyboardButton(text="🔍 Поиск номера")],
            [KeyboardButton(text="📅 Мои бронирования")],
            [KeyboardButton(text="➕ Добавить номер")],
            [KeyboardButton(text="❌ Отменить бронирование")],
            [KeyboardButton(text="ℹ️ Помощь")],
        ],
        resize_keyboard=True,
    )


def room_types_keyboard(room_types) -> InlineKeyboardMarkup:
    """Inline-клавиатура выбора типа номера (callback: rtype:<id>)."""
    buttons = [
        [InlineKeyboardButton(text=t.name, callback_data=f"rtype:{t.id}")]
        for t in room_types
    ]
    return InlineKeyboardMarkup(inline_keyboard=buttons)


def free_rooms_keyboard(rooms) -> InlineKeyboardMarkup:
    """Inline-клавиатура свободных номеров (callback: book:<room_id>)."""
    buttons = [
        [InlineKeyboardButton(text=f"{r.number} — {r.price} ₽/ночь",
                              callback_data=f"book:{r.id}")]
        for r in rooms
    ]
    return InlineKeyboardMarkup(inline_keyboard=buttons)


def booking_cancel_keyboard(bookings) -> InlineKeyboardMarkup:
    """Inline-клавиатура отмены бронирования (callback: cancel:<id>)."""
```

```
    buttons = [
        [InlineKeyboardButton(text=f"№{b.id}: {b.room.number} — {b.check_in}",
                              callback_data=f"cancel:{b.id}")]
        for b in bookings
    ]
    return InlineKeyboardMarkup(inline_keyboard=buttons)


def cancel_operation_keyboard() -> ReplyKeyboardMarkup:
    """Клавиатура с командой отмены FSM."""
    return ReplyKeyboardMarkup(
        keyboard=[[KeyboardButton(text="/cancel")]],
        resize_keyboard=True,
    )
```

8. FSM — многошаговые диалоги

Реализованы два FSM-сценария. Добавление номера (5 состояний): номер → тип → цена → вместимость → подтверждение; на каждом шаге выполняется валидация ввода (непустой номер, существующий тип, положительная цена, целая положительная вместимость, подтверждение «да»); после подтверждения вызывается сервис, создающий номер в базе. Бронирование (3 состояния): выбор номера (callback) → дата заезда → дата выезда; проверяются корректность формата даты и что выезд позже заезда, занятость номера. Команда /cancel очищает состояние в любой момент.

Файл: states/registration.py

```
from aiogram.fsm.state import State, StatesGroup


class AddRoomStates(StatesGroup):
    """Многошаговый FSM-сценарий добавления номера: 5 состояний."""

    number = State()
    room_type = State()
    price = State()
    capacity = State()
    confirm = State()


class BookingStates(StatesGroup):
    """FSM-сценарий бронирования номера."""

    select_room = State()
    check_in = State()
    check_out = State()
```

Файл: handlers/crud.py (фрагмент FSM)

```
@router.message(Command("room_add"))
async def cmd_room_add(message: Message, state: FSMContext) -> None:
    if not is_admin(message.from_user.id):
        await message.answer("Добавление номеров доступно только
администратору.",
                             reply_markup=main_menu())
        return
    await state.set_state(AddRoomStates.number)
    await message.answer(
        "Добавление номера. Шаг 1/5: введите номер комнаты (или /cancel):",
        reply_markup=cancel_operation_keyboard(),
    )
```

```
@router.message(AddRoomStates.number, F.text)
async def fsm_number(message: Message, state: FSMContext) -> None:
    if not message.text.strip():
        await message.answer("Номер не может быть пустым, повторите:")
        return
    await state.update_data(number=message.text.strip())
    await state.set_state(AddRoomStates.room_type)
    db = get_db()
    keyboard = room_types_keyboard(HotelService(db.session()).room_types())
    await message.answer("Шаг 2/5: выберите тип номера:", reply_markup=keyboard)


@router.callback_query(AddRoomStates.room_type, F.data.startswith("rtype:"))
async def fsm_room_type(callback: CallbackQuery, state: FSMContext) -> None:
    room_type_id = int(callback.data.split(":")[1])
    await state.update_data(room_type_id=room_type_id)
    await state.set_state(AddRoomStates.price)
    await callback.message.answer("Шаг 3/5: введите цену за ночь (число):")
    await callback.answer()


@router.message(AddRoomStates.price, F.text)
async def fsm_price(message: Message, state: FSMContext) -> None:
    try:
        price = float(message.text.strip())
    except ValueError:
        await message.answer("Цена должна быть числом (например 3500),
повторите:")
        return
```

9. Валидация и обработка ошибок

Обрабатываются как минимум три отдельные ситуации некорректного ввода: неверный формат даты (ответ «Некорректная дата, повторите»); дата выезда раньше даты заезда; повторное бронирование занятого номера; не положительная цена. Бизнес-ошибки (BookingError, NotFoundError) перехватываются в обработчиках и выводятся пользователю без завершения бота; FSM при ошибке корректно завершается.

10. Middleware

Собственный middleware LoggingMiddleware (BaseMiddleware) логирует каждое входящее сообщение: идентификатор и имя пользователя, текст, количество обработанных сообщений и время обработки. Он регистрируется на уровне Dispatcher (dp.message.middleware(...)) и выполняется до и после обработчика для всех сообщений.

11. Работа с базой данных и асинхронность

Бизнес-логика полностью отделена от Telegram-обработчиков в HotelService: методы get_or_create_client, free_rooms, search_rooms, create_booking (с расчётом ночей и стоимости и переводом номера в статус

«занят»), cancel_booking (проверка принадлежности бронирования клиенту и освобождение номера), history, create_room/update_room/delete_room, statistics. Все обработчики объявлены async def и используют await для вызовов Telegram API; блокирующие операции в боте не используются, что позволяет обрабатывать множество диалогов конкурентно в одном event loop.

12. Демонстрация работы

Демонстрация работы бота (сценарий: /start → свободные номера → поиск → бронирование → история → отмена → добавление номера → статистика) приведена на рисунке 17.2.

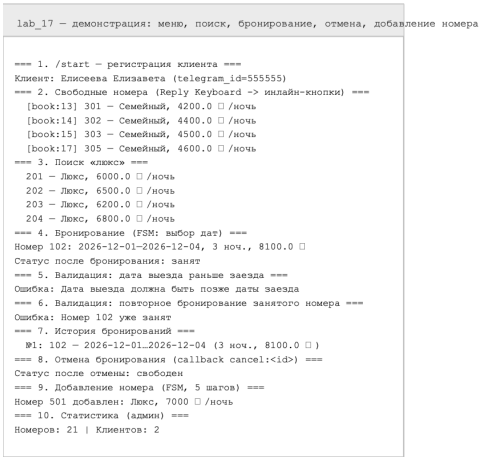


Рисунок 17.2 – Демонстрация сценария работы бота

13. Тестирование

Написано 22 автоматических теста pytest, покрывающих бизнес-логику (отдельно от Telegram-интерфейса): CRUD номеров (создание, получение, изменение, удаление, дубликат, неположительная цена/вместимость), поиск и свободные номера, бронирование (расчёт ночей и стоимости, ошибки дат,

занятый номер, история, отмена, отмена чужого бронирования) и клиенты (создание, статистика). Результат выполнения приведён на рисунке 17.3.

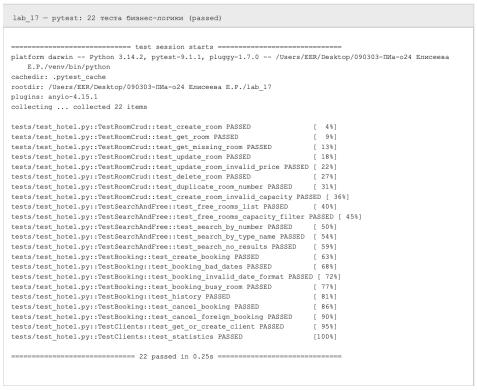


Рисунок 17.3 – Результаты выполнения pytest (22 passed)

14. Вывод

В ходе лабораторной работы разработан полноценный асинхронный Telegram-бот гостиницы на aiogram 3: реализованы команды и главное меню, Reply и Inline клавиатуры, обработка Callback Query, два FSM-сценария (бронирование и добавление номера), валидация ввода, работа с базой данных SQLAlchemy (4 сущности, 20 записей), поиск, отмена бронирования, история, middleware логирования и CRUD через Telegram.

Приложение разделено на модули (handlers, keyboards, states, services, database, middlewares), бизнес-логика вынесена в сервисный слой, токен хранится в переменной окружения, а 22 автоматических теста подтверждают корректность бизнес-логики. Получены практические навыки асинхронного программирования на asyncio, маршрутизации событий, работы с FSM и интеграции бота с базой данных, которые будут использоваться при изучении микросервисной архитектуры.